Python Programming Guide

Complete 10-Module Reference & Handcrafted Study Notes

Table of Contents (Anukramani)

1. Python Kya Hai?

Python ek high-level, interpreted, dynamically typed aur beginner-friendly programming language hai. Iska syntax saaf aur padhne me aasan (human-readable) hota hai.

2. First Python Program

Python me code run karna bohot simple hai. `print()` function ka use output dikhane ke liye hota hai.

```
# Yeh aapka pehla program hai
print("Namaste World! Welcome to Python.")

# Multiple lines print karne ke liye
print("Python seekhna aasan hai.")
```

3. Input and Output (I/O)

User se input lene ke liye `input()` function ka use kiya jata hai. By default, input hamesha **String** type me hota hai.

```
# User se input lena
name = input("Apna naam bataiye: ")
age = input("Apni umar bataiye: ")

# Formatted String (F-String) ka upayog
print(f"Aapka naam {name} hai aur aapki umar {age} saal hai.")
```

1. Variables & Data Types

Python me variables declare karne ke liye datatype likhna nahi padta. Values automatic dynamically bind ho jati hain.

Data Type	Description	Example
`int`	Purnaank (Whole numbers)	`x = 100`
`float`	Decimal numbers	`pi = 3.14159`
`str`	Text characters	`city = "Mumbai"`
`bool`	True ya False	`is_passed = True`

2. Type Casting

Ek datatype ko doosre datatype me badalna Type Casting kehlata hai.

```
str_num = "50"
int_num = int(str_num)   # String to Integer
float_num = float(int_num) # Integer to Float

print(type(float_num))   # Output: <class 'float'>
```

3. Operators Summary

```
a, b = 10, 3

# Arithmetic Operators
print(a + b)   # 13 (Addition)
print(a / b)   # 3.333... (Float Division)
print(a // b)  # 3 (Floor Division)
print(a ** b)  # 1000 (Power/Exponent)

# Comparison & Logical
print(a > b and b > 0) # Output: True
```

1. Conditional Statements (if, elif, else)

Program me decisions lene ke liye Conditional Statements ka use hota hai. Python me indentation (spacing) bohot zaroori hoti hai.

```
marks = 85

if marks >= 90:
    grade = "A+"
elif marks >= 75:
    grade = "A"
elif marks >= 60:
    grade = "B"
else:
    grade = "C"

print(f"Aapka Grade hai: {grade}")
```

Important Note: Indentation Rule

Python me `{ }` brackets ki jagah 4 spaces (Indentation) ka upayog code blocks define karne ke liye kiya jata hai.

2. Short-hand IF / Ternary Operator

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)  # Output: Adult
```

1. For Loop

Jab iterations ka number pehle se pata ho tab `for` loop use karte hain.

```
# range(start, stop, step)
for i in range(1, 6):
    print(f"Iteration Number: {i}")

# Strings par loop
for char in "PYTHON":
    print(char, end="-")  # Output: P-Y-T-H-O-N-
```

2. While Loop

Jab tak condition True hai tab tak `while` loop chalta rehta hai.

```
count = 3
while count > 0:
    print(f"Countdown: {count}")
    count -= 1
print("Blast Off!")
```

3. Break, Continue & Else in Loops

```
for num in range(1, 10):
    if num == 5:
        break     # Loop se bahar nikal jayega
    if num % 2 == 0:
        continue  # Curren iteration skip karega
    print(num)
```

1. Lists (Mutable / Badla ja sakta hai)

List ordered elements ka collection hota hai jisme alag-alag data types store ho sakte hain.

```
fruits = ["Apple", "Banana", "Mango"]

# Modifying List
fruits.append("Orange")      # Element add karega end me
fruits.insert(1, "Kiwi")      # Specific index par add karega
fruits.remove("Banana")      # Element remove karega

print(fruits[0])             # Output: Apple (Indexing)
print(fruits[1:3])           # Slicing: ['Kiwi', 'Mango']
```

2. Tuples (Immutable / Badla nahi ja sakta)

Tuples fast hote hain aur inki values fix rehti hain.

```
dimensions = (1920, 1080)
# dimensions[0] = 1280  --> Error dega! Tuples change nahi hote.

# Unpacking Tuples
width, height = dimensions
print(f"Width: {width}, Height: {height}")
```

1. Dictionaries (Key-Value Pairs)

Fast data retrieval ke liye Dictionaries ka use hota hai.

```
student = {
    "roll_no": 101,
    "name": "Rahul Sharma",
    "subjects": ["Maths", "Physics"]
}

# Access & Update
print(student["name"])           # Output: Rahul Sharma
student["grade"] = "A"           # New key-value add karna
student["roll_no"] = 102         # Value update karna

# Loop through Dictionary
for key, value in student.items():
    print(f"{key}: {value}")
```

2. Sets (Unique & Unordered Elements)

Sets me duplicate values store nahi hoti.

```
unique_ids = {101, 102, 103, 101}
print(unique_ids)  # Output: {101, 102, 103}

# Set Operations
set_a = {1, 2, 3}
set_b = {3, 4, 5}
print(set_a.union(set_b))        # Output: {1, 2, 3, 4, 5}
print(set_a.intersection(set_b)) # Output: {3}
```

1. Defining Functions

Code reusability ke liye Functions likhe jate hain. `def` keyword se function banta hai.

```
def calculate_total(price, tax=0.05):
    total = price + (price * tax)
    return total

# Function Call
bill1 = calculate_total(100)       # Uses default tax
bill2 = calculate_total(100, 0.18) # Uses 0.18 tax
print(f"Bill 1: {bill1}, Bill 2: {bill2}")
```

2. *args and **kwargs

Variable number of arguments pass karne ke liye use hote hain.

```
def add_all(*numbers):
    return sum(numbers)

print(add_all(10, 20, 30, 40)) # Output: 100
```

3. Lambda (Anonymous) Functions

Ek line ke chote functions likhne ke liye Lambda use hota hai.

```
square = lambda x: x * x
print(square(6)) # Output: 36
```

1. Class and Objects

Class ek blueprint hota hai aur Object uska instance hota hai.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner          # Attribute
        self.balance = balance

    def deposit(self, amount):      # Method
        self.balance += amount
        print(f"Deposited {amount}. New Balance: {self.balance}")

# Creating Object
acc1 = BankAccount("Amit", 1000)
acc1.deposit(500)
```

2. Inheritance (Virasat)

Ek class ki properties doosri class me inherit karna.

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):  # Dog inherits Animal
    def speak(self):
        print("Dog barks!")

my_dog = Dog()
my_dog.speak()  # Output: Dog barks!
```

1. Exception Handling (Try / Except)

Errors ko gracefully handle karne ke liye use hota hai taaki program crash na ho.

```
try:
    num1 = int(input("Enter number 1: "))
    num2 = int(input("Enter number 2: "))
    result = num1 / num2
    print(f"Result: {result}")
except ZeroDivisionError:
    print("Error: Zero se divide nahi kar sakte!")
except ValueError:
    print("Error: Kirpya valid number hi daalein.")
finally:
    print("Execution Completed.")
```

2. File Handling

Files me read aur write karne ke liye `with open()` sabse safe tarika hai.

```
# File Write karna
with open("notes.txt", "w") as f:
    f.write("Python seekhna bohot aasan aur majedar hai!")

# File Read karna
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

1. Essential Built-in Modules

```
import math
import random
import datetime

print(math.sqrt(64))                    # Output: 8.0
print(random.randint(1, 100))           # Random number between 1 & 100
print(datetime.date.today())            # Current Date
```

2. Python Best Practices (PEP 8 Highlights)

- **Variable Names:** Variable aur Function names ke liye `snake_case` use karein (e.g. `user_age`).
- **Class Names:** Class names ke liye `PascalCase` use karein (e.g. `UserProfile`).
- **Comments:** Code ke kaam ko samjhane ke liye clear aur meaningful comments (`#`) ka upayog karein.
- **DRY Principle:** "Don't Repeat Yourself" — Repeat hone wale code ko functions me convert karein.

Congratulations!

Aapne Python ka 10-Module core course complete kar liya hai! Regular practice aur small projects banakar aap isme master ban sakte hain.